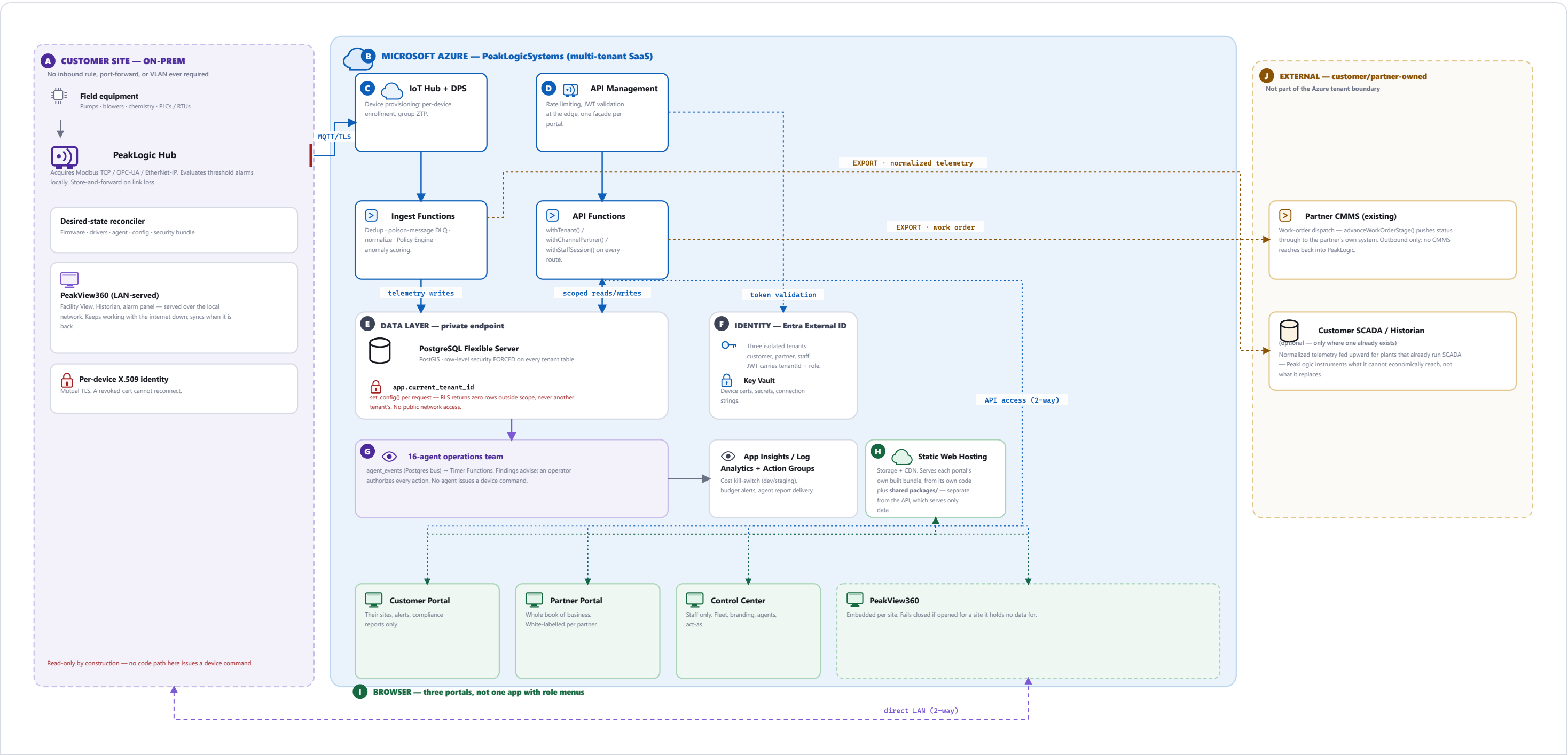


PeakLogic platform — system & data-flow diagram

Azure-native, multi-tenant. Every arrow below is a real data flow; every dashed boundary is an enforced isolation boundary, not an aspiration. Prepared for architectural and security review.

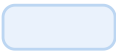


Reading the diagram

A fixed vocabulary of shapes, used consistently, plus a lettered badge on every boundary — cross-referenced below — so a review conversation can say "look at E" instead of re-pointing at the screen. One honest limitation: these are hand-drawn analogues of the well-known Azure shapes, not Microsoft's own licensed icon set, which this document's build process has no way to embed.

Boundaries

**On-prem boundary** — the customer's own network. Nothing inside requires an inbound rule from the cloud.

**Azure tenant boundary** — every managed service PeakLogic operates.

**Private sub-boundary** — data layer (no public network access) and identity.

**External ownership boundary** — the partner's CMMS, a customer's own SCADA. Not part of the Azure tenant.

Icons

**Gateway** — PeakLogic Hub, on-prem edge acquisition

**Managed cloud service** — Azure PaaS component

**Database** — PostgreSQL Flexible Server

**Serverless function** — Azure Functions

**Security control** — identity, encryption, secrets

**User-facing surface** — a portal or operator view

Flow lines

**Solid** — telemetry / data write path, always active, one direction only

**Dashed, both ends arrowed** — a genuine request/response round trip (browser↔API, LAN path)

**Dashed, one arrow** — a one-way check, e.g. token validation

**Green dashed** — static-asset delivery (SPA hosting), distinct from API data calls

**Amber dashed** — export to a system the customer or partner already owns, deliberately one-way by design

**Blocked stub** — a path that does not exist today (see Roadmap)

Domain reference

Every lettered badge on page 1, in one place.

A Customer site (on-prem)
Field equipment, PeakLogic Hub, LAN-served PeakView360

F Identity
Entra External ID (3 tenants) + Key Vault

B Microsoft Azure
The overall multi-tenant SaaS boundary — everything below is inside it except J

G Operations
16-agent team + App Insights/Log Analytics/Action Groups

C Device ingest
IoT Hub/DPS + Ingest Functions — provisioning and telemetry

H Static hosting
Storage + CDN serving the built SPA bundle

D API layer
API Management + API Functions — every portal request

I Browser surfaces
Customer Portal, Partner Portal, Control Center, PeakView360

E Data layer
PostgreSQL Flexible Server, RLS-forced, private endpoint

J External systems
Partner's CMMS, customer's own SCADA — not PeakLogic's

PeakLogic · Architecture Diagram v1.0 · 9 August 2026

Page 2 of 5

What a reviewer should take away

FLOW	PATH	SECURITY CONTROL
Telemetry ingest	Equipment → Hub → IoT Hub/DPS → Ingest Functions → PostgreSQL	MTLS per-device X.509 cert, outbound-only session, no inbound firewall rule ever required on the customer network. One direction only
Operator / customer request	Browser ⇌ APIM ⇌ API Functions ⇌ PostgreSQL	ENTRA JWT validated at APIM; every route wrapped in withTenant() / withChannelPartner() / withStaffSession(), which activates PostgreSQL row-level security. Genuine round trip — request and response
Static asset delivery	Browser → Storage + CDN (Static Web Hosting)	Separate from the API path above — this serves the SPA's code, not its data. No auth gate needed; nothing here is tenant-scoped
PeakView360 (embedded)	Portal iframe ⇌ Hub, direct over the site LAN, when opened on-site	Fail-closed scope guard — refuses to render if it holds no data for the requested site, rather than falling back to another site's facility. Two-way
Agent operations	PostgreSQL event bus → Timer Functions → Action Groups (email) / audit log	Findings carry tenant_id and are RLS-scoped. Agents propose only — every action needs operator authorization, logged to that operator, never to the agent
Work-order dispatch (export)	API Functions → PARTNER CMMS , outbound only	advanceWorkOrderStage() pushes ticket status through to the partner's own CMMS. Nothing in that system calls back into PeakLogic — the trust direction is one-way
Telemetry export (optional)	Ingest Functions → CUSTOMER SCADA/HISTORIAN , where one already exists	Normalized telemetry only, fed upward — PeakLogic instruments the long tail an existing SCADA cannot economically reach, it does not replace safety-rated control logic
Command / actuation	NOT BUILT	No code path publishes a device command today. The network model (outbound MQTT/TLS, per-device topic) already supports it; authorization, safety interlocks and audit trail are designed before any command path is opened

Tenant isolation

Every tenant-scoped query runs inside a transaction that sets `app.current_tenant_id` via `set_config()`, activating PostgreSQL RLS — **forced**, not merely enabled, and with no `BYPASSRLS` role. A site outside a viewer's scope is refused, never approximated.

Deployment reality

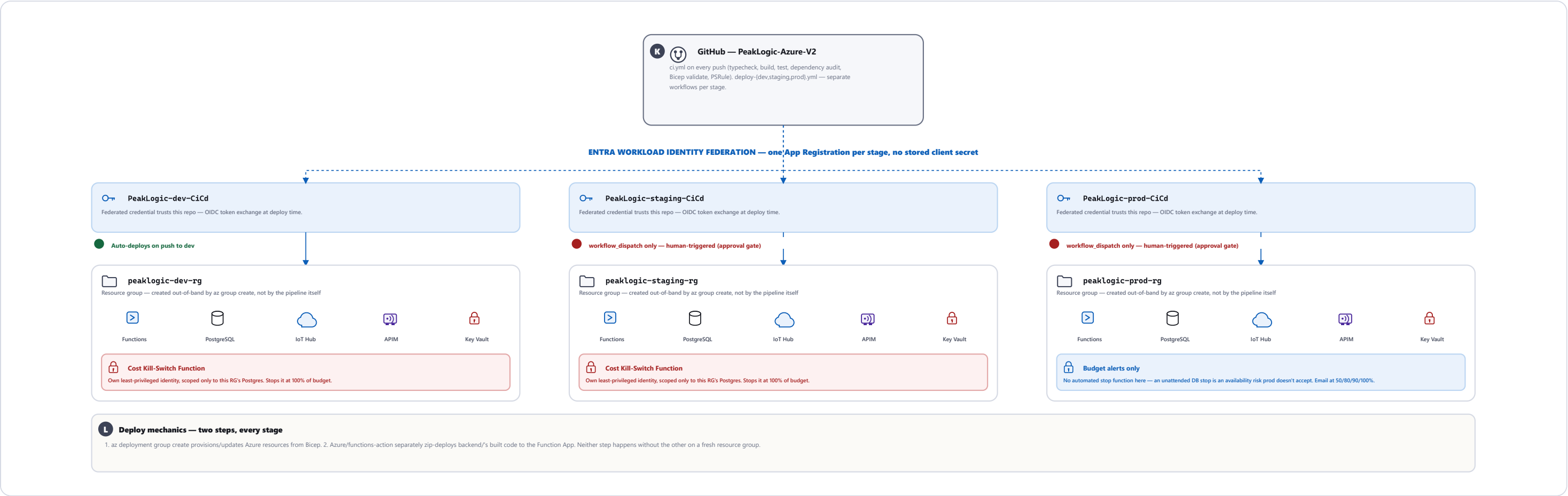
Defined in Bicep, validated by `az bicep build` and `PSRule` on every push. **NO ENVIRONMENT DEPLOYED YET** — this diagram describes the architecture as designed and CI-validated, not a running production system.

Cost containment

Dev and staging each carry a dedicated Cost Kill-Switch Function (its own least-privileged identity, scoped only to that stage's Postgres resource) that stops the database at 100% of budget. Prod deliberately does not carry this function — an automated stop is an availability risk that environment doesn't accept — and instead gets a real four-threshold budget routed entirely to email. Agents run on the existing consumption plan — additive spend is near \$0.

How the Azure tenant itself is organised

What this diagram shows — the deployment path: how code in source control becomes running Azure infrastructure, and what separates dev from production. Top to bottom: **(K)** the GitHub repository (CI on every push, one deploy workflow per stage) → **Entra Workload Identity Federation**, issuing each stage its own App Registration that trusts this repo over OIDC, so no client secret is ever stored → three **resource groups**, one per stage, each with a full copy of the platform's services and its own budget guardrail → **(L)** the two-step mechanism every deploy performs. The coloured dot above each resource group is its trigger: dev deploys automatically; staging and prod need a human to start the run. Dev and staging carry an automated Cost Kill-Switch Function that can stop their own database at 100% of budget; prod deliberately does not — see the card below. Pages 1–3 cover how data moves through the running system; this page covers how that system gets deployed.



NOT DEPLOYED No environment has been created yet — peaklogic-{dev,staging,prod}-rg, the Entra App Registrations, and the federated credentials are all still to-do, not running infrastructure. This page describes the designed target, same disclosure as pages 1–3.

How this actually scales, and how ready it is today

Pages 1–4 show what is built and how it is secured. This page answers the two questions those pages don't: **at what real, numeric scale does this design need to change**, and **how ready is it to run in production right now** — scored by an independent third-party Principal Architect review commissioned against the full 45-document architecture corpus, not this diagram alone.

Scale triggers — when the design changes, not if

Numeric thresholds already defined in the platform's own compute forecast, re-verified independently. **No new infrastructure is justified before these are hit** — not Kafka, not Cosmos DB, not a dedicated message broker, not a physical microservice split.

SITES	TRIGGER	ACTION
~250	PostgreSQL Burstable tier saturates	Move off Burstable — the first real infrastructure decision this platform will make
~2,000	Vacuum/full-scan cost on the telemetry table starts to hurt	Partition telemetry by month — already a -- partition by month in v2 comment in the live schema, not a future redesign
~2,778	IoT Hub S1 unit capacity reached	Move to S2; evaluate consuming Event Hubs directly for the highest-volume sites, bypassing IoT Hub's per-device features where they're no longer needed

Independent scorecard — representative areas

AQ = Architecture Quality (is the design sound) · **IM** = Implementation Maturity (is it actually built and verified) · **OR** = Operational Readiness (could this safely run in production today). Scored 0–10; full 25-area scorecard in the underlying review.

AREA	AQ	IM	OR	INDEPENDENT REVIEWER'S NOTE
Overall architecture	8	5	3	Strong corpus, patchy build, nothing deployed
Security	8	6	3	Six "closed" findings all carry disclosed residuals; zero real-subscription validation
Multi-tenancy	8	7	5	Mechanism now correct; test-proof gap (Gap 1) closed 2026-08-09
Testing	8	8	6	Genuinely strong — badly undersold by the project's own docs until corrected
Cost architecture	6	7	4	Clever, fully wired; binary kill-switch replaced with a graduated tier 2026-08-09
CI/CD	7.5	7	3	One of the stronger areas; real tests, real scanning; never deployed live
Scalability	7	4	2	Honest, numeric scale triggers; the code those triggers point to mostly waits on real traffic
Infrastructure	7	5	2	Well-reasoned Bicep modules; zero live-subscription validation anywhere

NOT MARKETING The AQ/IM/OR gap above is the point of this page, not a flaw to hide: a platform that honestly scores itself 8/5/3 and shows exactly which three conditions it closed and why the fourth can't be closed without a live deployment is a stronger signal of engineering discipline than a diagram with no gap on it at all.

Reviewer's verdict: conditional sign-off

"*The foundation is sound. It needs the last mile of proof, not more design.*" Four concrete, boundable conditions were named — none requiring new infrastructure or a redesign:

RLS isolation proven by a superuser test connection that bypasses it by definition, not the real app role	✓ CLOSED 2026-08-09
Tenant-side users have no per-site authorization scope, unlike the partner side's real DB-enforced territory model	✓ CLOSED 2026-08-09
CLAUDE.md stated "no test suites exist," contradicted by 38 real test files already in the repo	✓ CLOSED 2026-08-09
Nothing has ever been deployed to a live Azure subscription — every scale/alerting/rollback claim is unverified against reality	OPEN — SEQUENCING FACT, NOT A DESIGN DEFECT (PAGE 4)